Vulnerability Assessment: **CAPTCHA Bypass / Insecure CAPTCHA Enforcement**
Project 4: Web Application Security Assessment
Week 3: Secure SDLC and Code Review

Vulnerability Information

- **Vulnerability Type:** CAPTCHA Bypass / Insecure Verification
- **Risk Level:** High
- **OWASP Category:** A07:2021 – Identification and Authentication Failures
- **Location:** /vulnerabilities/captcha/, lines 1–8

Vulnerable Code Analysis

- Low

```
1  if( isset( $_POST[ 'Change' ] ) && ( $_POST[ 'step' ] == '2' ) ) {
2      // Hide the CAPTCHA form
3      $hide_form = true;
4
5      // Get input
6      $pass_new  = $_POST[ 'password_new' ];
7      $pass_conf = $_POST[ 'password_conf' ];
8
```

- Medium

```php
if( !$_POST[ 'passed_captcha' ] ) {
    $html     .= "<pre><br />You have not passed the CAPTCHA.</pre>";
    $hide_form = false;
    return;
}
```

- High

```php
if (
    $resp ||
    (
        $_POST[ 'g-recaptcha-response' ] == 'hidd3n_valu3'
        && $_SERVER[ 'HTTP_USER_AGENT' ] == 'reCAPTCHA'
    )
){
```

The CAPTCHA implementation relies on **client-side values** (passed_captcha, g-recaptcha-response) and **hidden fields**, which attackers can easily manipulate.
There is **no server-side verification** of the CAPTCHA response token or session-based tracking.
Attackers can skip CAPTCHA validation by sending a direct POST request with step=2 and passed_captcha=true, completely bypassing human verification.

Secure Code Screenshot:

```php
session_start();

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['Change']) && $_POST['step'] === '2') {
    // Enforce server-side CAPTCHA validation
    if (empty($_SESSION['captcha_passed']) || $_SESSION['captcha_passed'] !== true) {
        http_response_code(400);
        echo "CAPTCHA verification required.";
        exit;
    }

    $pass_new  = $_POST['password_new'];
    $pass_conf = $_POST['password_conf'];
}

// CAPTCHA verification endpoint
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['captcha_token'])) {
    $token = $_POST['captcha_token'];
    $resp = verify_recaptcha($token);

    if ($resp && $resp['success'] === true) {
        $_SESSION['captcha_passed'] = true;
        echo json_encode(['ok' => true]);
    } else {
        $_SESSION['captcha_passed'] = false;
        echo json_encode(['ok' => false]);
    }
}
```

Mitigations Applied:

- Implemented **server-side CAPTCHA validation** using provider API verification
- Removed reliance on client-supplied flags like passed_captcha
- Stored CAPTCHA status securely in **server session**
- Enforced CAPTCHA verification before continuing with sensitive actions
- Required **POST requests** and validated workflow steps on the server

Recommendations for Development Team

Immediate Actions:

1. **Server-Side Verification**
    - Always validate CAPTCHA tokens server-side via official provider API (e.g., Google reCAPTCHA).
    - Reject requests that skip verification or reuse tokens.
2. **Session-Based Tracking**
    - Store CAPTCHA verification results in the user's session.
    - Ensure CAPTCHA completion is tied to the same session that initiated the request.
3. **Prevent Direct Access to Steps**
    - Validate the form submission sequence server-side (step=1 → step=2).
    - Reject any direct access to later stages without valid session data.
4. **Rate Limiting and Monitoring**
    - Add rate limits per IP and user to prevent automated CAPTCHA abuse.
    - Log failed CAPTCHA attempts and analyze patterns of automated activity.

References

- **OWASP Cheat Sheet:** Authentication Cheat Sheet
- **CWE Reference:** CWE-804: Guessable CAPTCHA
- **Best Practice:** Use server-verified CAPTCHA tokens, session-based tracking, and strict form sequence validation to prevent CAPTCHA bypass.